HW1:

What is the output of the following code?

```
#include <iostream>

class Base {
    virtual void method() { std::cout << "from Base" << std::endl; }
public:
    virtual ~Base() { method(); }
    void baseMethod() { method(); }
};

class A : public Base {
    void method() { std::cout << "from A" << std::endl; }
public:
    ~A() { method(); }
};

int main(void) {
    Base* base = new A();
    base->baseMethod();
    delete base;
    return 0;
}
```

from A

from A

from base

HW2:

What is the output of the program!! And please explain what is going on?

```
#include <iostream>

class Base
{
    public:
        virtual void foo() const { std::cout << "A's foo!" << std::endl; }
};

class Derived : public Base
{
    public:
        void foo() { std::cout << "B's foo!" << std::endl; }
};

int main()
{
    Base* o1 = new Base();
    Base* o2 = new Derived();
    Derived* o3 = new Derived();

    o1->foo();
    o2->foo();
    o3->foo();
}
```

A’s foo!

A’s foo!

B’s foo!

Explain: o1 就是正常的印出 Base class 的 foo func；o2 是多型寫法，所以 compiler 會先到 vtable 去查 vptr，但會發現 Derive 根本沒有 void foo() const {…}的 func，所以會指向失敗，印出在 Base 的 foo func，要加 override 才能報錯；而 o3 在 compile time 就焊死在 Derive class 了，所以跟 o1 一樣，也不需要查 vtable，直接用 Derive func

HW3:

請以多型完成下列程式碼，演奏弦樂四重奏。

```
public class V {
	public static void main(String[] args) {
		// 弦樂四重奏
		Instrument[] stringQuartet = { new Violin(), new Violin(), new Viola(), new
Cello() };
		// play the music for me
		// implement HERE !!!
	}
}

abstract class Instrument {
	abstract public void play();
}

// 小提琴
class Violin extends Instrument {
	@Override
	public void play() {
		System.out.println("旋律");
	}
}

// 中提琴
class Viola extends Instrument {
	@Override
	public void play() {
		System.out.println("合旋");

	}
}

// 大提琴
class Cello extends Instrument {
	@Override
	public void play() {
		System.out.println("低音");

	}
}
```

ANS:

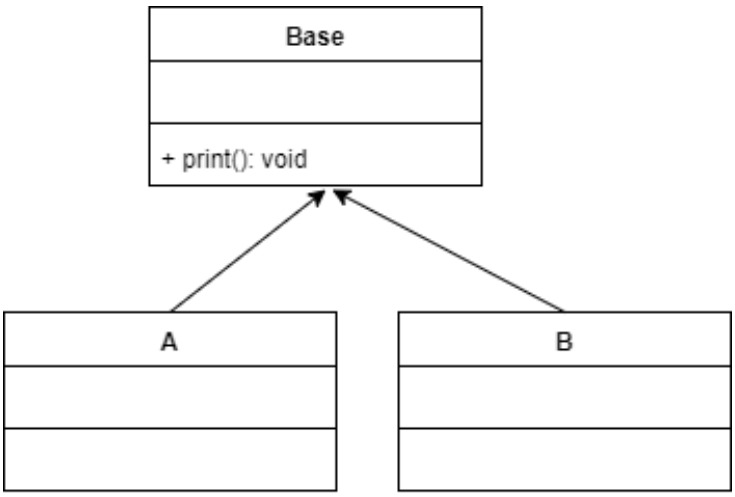
```
public class V {
        public static void main(String[] args) {
                Instrument[] stringQuartet = { new Violin(), new Violin(), new Viola(), new
Cello() };
      // add here
      for (Instrument i : stringQuartet) {
         i.play();
      }
         }
}
```

HW4:

Please implement (use polymorphism) the following code to make result generate the same output.

```
#include <iostream>
#include <vector>

using namespace std;

class Base {
public:
   virtual void print() { cout << "Base" << endl; }
   virtual ~Base() {}
};

int main(void) {
   vector<Base*> bases = { new Base(), new A(), new B() };
   for (Base* b : bases) {
      b->print();
   }

   for (Base* b : bases) {
      delete b;
      b = nullptr;
   }
   bases.clear();
}
```

Please make inherit fit following class diagram.



Output:

```
$ ./a.exe
Base
A
B
```

ANS:

```
#include <iostream>
#include <vector>

using namespace std;

class Base {
public:
    virtual void print() { cout << "Base" << endl; }
    virtual ~Base() {}
};

class A: public Base {
    public:
    void print() {cout << "A\n";}
    ~A() {}
};

class B: public Base {
    public:
    void print() {cout << "B\n";}
    ~B() {}
};

int main(void) {
    vector<Base*> bases = { new Base(), new A(), new B() };
    for (Base* b : bases) {
        b->print();
    }

    for (Base* b : bases) {
        delete b;
        b = nullptr;
    }
    bases.clear();
}
```

HW5:

Modify the attachment by using polymorphism.

```
#include <stdio.h>
#include <assert.h>

#define SIZE 3

typedef enum {
    Comic,
    Novel,
    Magazine
} book_type;

typedef struct Book {
    book_type type;
} Book;

void print_book_type(Book*);

int main(void) {
    Book books[SIZE];
    books[0].type = Comic;
    books[1].type = Novel;
    books[2].type = Magazine;

    for (int i = 0; i < SIZE; i++) {
        print_book_type(&books[i]);
    }

    return 0;
}

void print_book_type(Book* book) {
    assert(book);

    switch (book->type) {
    case Comic:
        printf("Comic\n");
        break;
    case Novel:
        printf("Novel\n");
        break;
    case Magazine:
        printf("Maganize\n");
        break;
    }
}
```

ANS:

```
#include <stdio.h>
#include <assert.h>
#include<iostream>
#include<vector>
#define SIZE 3
using namespace std;
class Book {
   public:
   virtual void print_book() = 0;
   virtual ~Book() {}
};

class Comic: public Book {
   public:
   void print_book() {
      cout << "Comic\n";
   }
   ~Comic() {}
};

class Novel: public Book {
   public:
   void print_book() {
      cout << "Novel\n";
   }
   ~Novel() {}
};

class Magazine: public Book {
   public:
   void print_book() {
      cout << "Magazine\n";
   }
   ~Magazine() {}
};

int main() {
   vector<Book*> list = {new Comic(), new Novel(), new Magazine()};
   for (auto& type: list) type->print_book();
   for (auto& i: list) {
```

```
        delete i;
    }
}
```

HW6:

Please write a main function (with polymorphism way) to make result generate the same output.

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void speak() = 0;
    virtual ~Animal() {}
};
class Cat : public Animal {
public:
    void speak() {
        cout << "meow" << endl;
    }
};
class Cow : public Animal {
public:
    void speak() {
        cout << "moo" << endl;
    }
};
```

Output:

```
$ ./animal.exe
meow
moo
```

ANS:

```
#include <iostream>
#include <vector>
using namespace std;

class Animal {
public:
    virtual void speak() = 0;
    virtual ~Animal() {}
```

```
};
class Cat : public Animal {
public:
    void speak() {
        cout << "meow" << endl;
    }
};
class Cow : public Animal {
public:
    void speak() {
        cout << "moo" << endl;
    }
};

int main() {
    vector<Animal*> animals = {new Cat(), new Cow()};
    for (auto& i: animals) i->speak();
    for (auto& i: animals) delete i;
}
```